

Hands-on example

Building a balanced research system

What you'll build: A research workflow that gathers perspectives from two sources in parallel, then combines them.

Step 1: Create the project

Shell

```
adk create balanced_research
cd balanced_research
```

Step 2: Write the agent

Replace `agent.py` with:

Python

```
"""
Balanced Research System
Demonstrates Sequential + Parallel workflow with proper state coordination.

Reference: https://google.github.io/adk-docs/agents/workflow-agents
"""

from google.adk.agents import LlmAgent, SequentialAgent, ParallelAgent

# =====
# PARALLEL STAGE: Two researchers run at the same time
# =====
benefits_researcher = LlmAgent(
    model='gemini-2.5-flash',
    name='benefits_researcher',
    output_key='benefits', # Distinct key
    instruction=''
    Research the BENEFITS of the given topic.
```



```
Provide:
```

- 3-5 key advantages
- Supporting evidence
- Real-world examples
- ...

```
)
```

```
challenges_researcher = LlmAgent(  
    model='gemini-2.5-flash',  
    name='challenges_researcher',  
    output_key='challenges', # Distinct key (different from above)  
    instruction='''  
    Research the CHALLENGES of the given topic.  

```

```
    Provide:
```

- 3-5 key challenges
- Why these matter
- Potential mitigations
- ...

```
)
```

```
parallel_research = ParallelAgent(  
    name='parallel_research',  
    sub_agents=[benefits_researcher, challenges_researcher]  
)
```

```
# =====  
# SEQUENTIAL STAGE: Aggregator combines parallel results  
# =====
```

```
summarizer = LlmAgent(  
    model='gemini-2.5-flash',  
    name='summarizer',  
    instruction='''  
    Create a balanced summary using both research perspectives:
```

```
    BENEFITS:
```

```
    {benefits}
```

```
    CHALLENGES:
```

```
    {challenges}
```

```
    Your summary should:
```

- Present both sides fairly
- Highlight trade-offs
- Provide a balanced conclusion
- ...

```
)
```




```
# =====
# COMPLETE WORKFLOW: Parallel research → Sequential summary
# =====
root_agent = SequentialAgent(
    name='balanced_research',
    sub_agents=[parallel_research, summarizer]
)
```

What this demonstrates:

- **ParallelAgent** runs both researchers simultaneously (faster)
- Distinct `output_key` values prevent state conflicts
- **SequentialAgent** ensures summarizer runs after parallel completes
- Aggregation pattern combines results via `{key}` templating

Step 3: Run and test

Shell
adk web

Navigate to <http://localhost:8000>.

Step 4: Test it

Test prompt:

You: `Research artificial intelligence in healthcare`

Expected behavior:

1. Benefits researcher and challenges researcher run in parallel
2. Both complete (notice they're faster together)
3. Summarizer combines both perspectives
4. Final output presents balanced view

What to notice:

- Both researchers run at the same time (parallel)
- No data is lost (distinct `output_key` values)
- Summarizer has access to both results
- Workflow is efficient and organized



Key takeaways

The patterns in this part form the foundation for complex workflow orchestration. Understanding when to use each workflow agent and how to coordinate state in parallel execution will help you build efficient multi-agent systems.

Choosing workflow agents

Use Case	Agent	Key Benefit
Ordered steps	SequentialAgent	Guaranteed order
Independent tasks	ParallelAgent	Faster execution
Iterative refinement	LoopAgent	Repeat until done

Parallel state coordination

- Always use distinct `output_key` for each parallel agent
- Follow with aggregator to combine results
- Wrap in `SequentialAgent` to ensure proper ordering

The essential pattern

```
Python
# Parallel agents with distinct keys
parallel = ParallelAgent(sub_agents=[
    LlmAgent(output_key='result_a', ...),
    LlmAgent(output_key='result_b', ...)
])

# Aggregator reads all results
aggregator = LlmAgent(instruction='Use {result_a} and {result_b}')

# Sequential wrapper
workflow = SequentialAgent(sub_agents=[parallel, aggregator])
```

